

Domain 2: Tool Design & MCP Integration

Domain 2 makes up 18% of the scored content, and it's where you learn to give Claude reliable hands in the outside world. You'll cover how to design tools that Claude reaches for correctly and uses reliably, how to write structured error responses that help an agent recover when something goes wrong, how to spread tools across agents so each one has a clean, focused job, how to bring MCP servers into your Claude workflows, and how to pick the right built-in tool for the task in front of you.

Think of tool design as the connection between Claude's reasoning and everything beyond it. When tools are poorly designed, you get misrouted calls, silent failures, and agents you can't trust. Get the tools right, and the rest of the system has something solid to stand on. So, what actually happens when Claude uses a tool?

A. Foundations of Tool Design for Claude

A tool call is an action Claude can request during a conversation. Claude does not execute tools directly, it sends a structured request containing the tool name and input parameters, and your code decides whether to execute, reject, or modify the request. This separation between tool selection and tool execution is fundamental to understanding how tool design affects agent reliability.

Claude uses tool names, descriptions, and input schemas to decide which tool is appropriate for the current step. When a user says "look up this order," Claude does not search your codebase for a function named `lookupOrder`. It reads the descriptions of all available tools and selects the one whose description best matches the intent. This means that a tool's description is not documentation for humans but rather a decision signal for Claude.

How Claude Selects Tools

Claude's tool selection works in three phases: first, on each turn, Claude reads all available tool descriptions, evaluates which one (if any) matches the current task, and either calls a tool or responds directly. The selection is driven by the match between the task context and the tool description, not by the tool name alone.

This demonstrates the following:

- A clear, specific description makes the right tool obvious to Claude.
- An ambiguous description makes Claude guess, producing inconsistent tool selection.
- Two tools with similar descriptions create confusion, as Claude may alternate between them randomly.
- A tool with a misleading name but a good description will still be selected correctly, because Claude weighs the description more heavily than the name.

Three Components of a Tool Definition

A tool definition has three parts, and each serves a distinct purpose.

Component	Purpose	Impact on Reliability
Name	A short identifier for the tool	Minor: Claude uses descriptions more than names for selection
Description	Tells Claude when and why to use this tool	Major: the primary signal for tool selection
Input schema	Defines what parameters the tool accepts	Major: determines whether Claude can construct a valid call

Why Tool Design Is an Architectural Concern

Tool design is about more than labeling, as the choices you make here drive how reliably the whole agent system works.

- A well-designed tool set makes Claude's choices predictable and testable.
- A badly designed one can introduce misrouting, duplicate calls, and cascading errors that no prompt engineering can fix.
- Tool descriptions also interact with the system prompt, so one stray keyword can push Claude towards a wrong tool.

EXAM TIP: The exam tests tool design as an architectural concern. When a question describes Claude calling the wrong tool, the first thing to check is the tool descriptions, not the prompt, not the model, and not the conversation history. Tool descriptions are the primary signal for tool selection.

Common Mistakes

- Writing tool descriptions for a human reader when Claude is the one acting on them.
- Trusting the tool name to carry the meaning while Claude reads the description.
- Creating tools with overlapping scopes and near-identical descriptions Claude can't tell apart.
- Forgetting that a keyword in your system prompt can quietly tip tool selection.

Resources

- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/overview>
- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/best-practices>

B. Designing Effective Tool Descriptions and Boundaries

The tool description is the most important part of a tool definition because it is the text Claude uses to determine whether a tool is appropriate for the current step. When the description is vague, Claude is forced to guess. Similarly, when the description overlaps with another tool's scope, it can lead to misrouting. By contrast, a clear and specific description explains what the tool does, when to use it, and how it differs from related tools, helping Claude consistently select the best option.

Anthropic's best practices on tool-use states that tool descriptions should clearly explain what the tool does, when it should be used, what each parameter means, and any important limitations. The documentation also states that tool descriptions are like good docstrings; the more context you provide, the better Claude can use them.

What Makes a Good Tool Description

A good tool description includes:

- 1. **Purpose:** A one-sentence explanation of what the tool does.
- 2. **Use case:** The specific conditions under which the tool should be used.
- 3. **Parameters:** A clear, plain-language explanation of every input field.
- 4. **Boundaries:** Explicit exclusions that prevent Claude from using the tool in the wrong situations.

WORKED EXAMPLE — Weak vs. Strong Tool Description

Weak: `name: "search" description: "Search for things"`

Strong: `name: "search_orders" description: "Look up customer orders by order ID, customer email, or date range. Use this when the customer asks about a specific order, order status, or order history. Returns order details including status, items, and shipping information. Do NOT use this for product searches, use search_products instead."`

What this shows: the strong description names the use cases (order lookup, status, history), specifies the input types (order ID, email, date range), describes the return value, and explicitly excludes product searches preventing misrouting to a similarly-named tool.

Preventing Misrouting Between Similar Tools

Misrouting happens when Claude picks the wrong tool, and the usual culprit is a pair of tools whose descriptions overlap or blur together. The fix is a disambiguation pattern, when two tools serve similar but distinct purposes, each description should spell out what the other one is for, so Claude can tell them apart.

Scenario	Without Disambiguation	With Disambiguation
Search orders vs. search products	"Search for orders" / "Search for products"	"Search for orders by ID, email, or date. Do NOT use for product catalog searches — use search_products." / "Search product catalog by name, category, or SKU. Do NOT use order lookups, use search_orders."
Get customer info vs. get customer orders	"Get customer information" / "Get customer orders"	"Get customer profile data: name, email, plan, account status. Does NOT return order history — use get_customer_orders." / "Get a customer's order history. Requires customer_id. Does NOT return profile data — use get_customer_info."

Tool Names vs. Tool Descriptions

A tool name is less important than the description, but a misleading one can still cause problems.

- Claude prefers description as its primary signal, so the name is always secondary.
- A name like `process_data` sitting next to a description about order lookups creates a dissonance that chips away at selection reliability.

- Keep names consistent with descriptions, so an order-search tool becomes `search_orders` rather than `query` or `lookup`.
- Steer clear of generic names like `helper`, `process`, `handle`, or `execute`, since they give Claude no signal about what the tool is for.

Parameter Descriptions

Each parameter in the input schema should have a description that tells Claude what to pass. Without parameter descriptions, Claude must infer the expected format from the parameter name alone, which can produce inconsistent inputs.

Parameter	Without Description	With Description
customer_id	(none)	"The unique customer identifier, e.g., 'C-1049'. Required for all customer-specific lookups."
date_range	(none)	"ISO 8601 date range as {start, end}. Both dates are inclusive. Maximum range: 90 days."
status_filter	(none)	"Filter by order status. Valid values: 'pending', 'shipped', 'delivered', 'cancelled'. Omit to return all statuses."

Tool Description Anti-Patterns

Anti-Pattern	Why it does not work	Fix
"Search for things"	Too vague, Claude cannot distinguish from other search tools	Name what is searched, what inputs are accepted, what is returned
No boundary statement	Claude may use this tool for tasks that belong to another tool	Add "Do NOT use for X, use tool_Y instead"
Technical jargon without context	Claude may misinterpret domain-specific terms	Use plain language or define terms in the description
Description contradicts the name	Creates confusion in tool selection	Align name and description
No parameter descriptions	Claude guesses at parameter formats	Add a description for every parameter

EXAM TIP: When a question describes Claude consistently calling the wrong tool, the answer is almost always to improve the tool descriptions. Add boundary statements ("Do NOT use for X"), add specificity about when to use the tool, and ensure there is no overlap with other tools' descriptions. Few-shot examples in the prompt do not fix tool misrouting, they fix output format and judgment, not tool selection.

Common Mistakes

- Writing descriptions for humans ("This tool searches orders") instead of for Claude ("Use this tool when the customer asks about order status, order history, or a specific order by ID").
- Creating two tools with overlapping scope and no disambiguation.

- Relying on the tool name to do the work of the description.
- Using few-shot examples to fix tool misrouting when the real cause is ambiguous descriptions.

Resources

- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/best-practices>
- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/overview>

C. Structured Error Responses for Tools

When a tool fails, the error message it returns determines whether the calling agent can recover. A generic error like "tool failed" or "error occurred" gives the agent no information to work with and it cannot decide whether to retry, try a different tool, or escalate. A structured error with a category, description, and retryability flag lets the agent make an informed recovery decision.

This section overlaps with Domain 5 (Error Propagation) but focuses specifically on how to design the error responses that tools return, rather than how agents propagate errors through multi-agent pipelines.

Problems with Generic Errors

When a tool returns a generic error, the calling agent has no basis for deciding what to do next.

- Should it retry? The error does not say whether the failure is transient.
- Should it try a different tool? The error does not say what went wrong.
- Should it escalate? The error does not say whether the situation is recoverable.
- Should it tell the user? The error does not contain enough detail for a meaningful message.

The result is that the agent either retries blindly (wasting tokens on a non-retryable error), gives up prematurely (missing a retry opportunity), or reports vague information to the user ("I encountered an error" which is unhelpful).

The Structured Error Pattern

A well-designed error response carries three fields that the agent can branch on.

Field	Purpose	Example Values
category	What type of failure occurred	"timeout", "permission_denied", "validation_error", "not_found", "rate_limited", "service_unavailable"
description	Human-readable detail of what went wrong	"Database connection timed out after 30s", "API key lacks read permission for billing records"
retryable	Whether the same call might succeed on retry	true / false

WORKED EXAMPLE

```
// Good: structured error
{
```

```
"is_error": true,
"category": "timeout",
"description": "Payment API did not respond within 30 seconds",
"retryable": true
}

// Bad: generic error
{
  "is_error": true,
  "message": "An error occurred"
}
```

Error Categories and Recovery Strategies

Different error categories require different recovery strategies. The agent's recovery logic should branch on the category, not on parsing the description text.

Category	Typical Cause	Retryable?	Recovery Strategy
timeout	Network latency, slow backend	Yes	Retry with exponential backoff
rate_limited	Too many requests	Yes	Wait, then retry after delay
permission_denied	Missing credentials or scope	No	Escalating as this cannot be fixed by retrying
not_found	Resource does not exist	No	This is information, and not an error, handled as valid empty
validation_error	Malformed request parameters	No	Fix the request parameters, then retry
service_unavailable	Backend is down	Yes	Try alternative source or annotate gap

Validation errors are retryable only if the agent can fix the input. If the input came from the user and is genuinely invalid, it is not retryable without user correction.

Access Failure vs. Valid Empty Result

- An **access failure** means the tool could not reach its data source. No query was executed. The absence of data is not meaningful.
- A **valid empty result** means the tool successfully queried its source and found nothing. The absence of data IS the answer.

If the system treats both the same way, downstream agents may report "no data exists" when the reality is "we could not check." Which is a silent data integrity failure.

Situation	What It Is	Correct Response
API returned HTTP 500	Access failure	Return error with category: "service_unavailable", retryable: true

Situation	What It Is	Correct Response
API returned HTTP 200 with empty array	Valid empty result	Return success with empty data
API connection timed out	Access failure	Return error with category: "timeout", retryable: true
API returned HTTP 404	Depends on context	If the resource should exist: error. If checking existence: valid empty.

WORKED EXAMPLE — MCP Tool Error Response

A customer lookup tool queries a database. The database connection times out.

Wrong pattern: Return an empty result. The coordinator concludes the customer does not exist. The agent tells the user "We could not find your account" — but the account exists, and the lookup just failed.

Correct pattern: Return: `{ "is_error": true, "category": "timeout", "description": "Customer database connection timed out after 10s", "retryable": true }` The coordinator sees a retryable timeout, waits, and retries. On the second attempt, the database responds, and the customer is found.

Returning Errors in MCP Tool Implementations

In MCP, tools return errors by setting `is_error: true` on the tool result. The content field carries the error details as text that Claude reads and reasons about.

Anthropic's MCP documentation explains that when `is_error` is true, Claude understands the tool call failed and can adapt its approach, such as retrying, trying a different tool, or reporting the issue to the user. Without the `is_error` flag, Claude may interpret error text as a normal tool result and try to reason over it as if it were valid data.

Error Response Design Principles

- **Always include a category.** This is what the agent branches on. Without it, the agent has to parse the description text, which is fragile and unreliable.
- **Always include retryability.** This prevents wasted retries on non-retryable errors and missed retry opportunities on transient failures.
- **Keep descriptions specific.** "Database connection timed out after 30s" is actionable. "An error occurred" is not.
- **Never return an empty result for an access failure.** This is the most dangerous anti-pattern because it looks like success.
- **Use categories consistently.** If every tool in your system uses the same category names, the coordinator can have a single recovery strategy for each category.

EXAM TIP: When a question describes a coordinator that "cannot determine what went wrong because the tool only returns a generic error message", the answer is structured error responses with category, description, and retryability. When a question describes a downstream agent that "incorrectly reports no data when the source was unreachable", the answer is distinguishing access failures from valid empty results.

Common Mistakes

- Returning empty results for tool failures, which silently corrupts downstream reasoning.
- Using generic error messages that give the agent no recovery information.
- Omitting the retryability flag, so the agent either retries everything or retries nothing.
- Treating HTTP 404 as always an error when it might be a valid "does not exist" answer.
- Using inconsistent error categories across tools, forcing the coordinator to handle each tool's errors differently.

References

- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/best-practices>
- <https://docs.anthropic.com/en/docs/agents-and-tools/mcp>
- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/implement-tool-use>

D. Tool Distribution Across Agents

In a multi-agent system, not every agent should have every tool. A search agent needs web search tools but does not need file write tools. A synthesis agent needs no search tools at all as it works from the outputs of other agents. Giving every agent every tool creates unnecessary decision complexity, increases the risk of inappropriate actions, and makes the system harder to audit.

Anthropic's Agent SDK documentation supports this through the `allowedTools` parameter, which restricts which tools a subagent can access. This is not just a security measure, as it improves tool selection reliability because Claude has fewer tools to choose from, making each decision simpler and more predictable.

The Principle of Least Privilege for Tools

Each agent should have access to only the tools necessary for its specific role. This principle mirrors security's principle of least privilege but is applied to tool access.

Reliability: When Claude has 15 tools available, it must evaluate all 15 descriptions on every turn to decide which one to use. With 3 tools, the decision space is much smaller, and selection is more reliable. Over-permissive tool sets do not just create security risk but also reliability risk.

Security: A subagent with write access to the database but whose job is only to read data can accidentally make destructive changes. Restricting its tools to read-only operations prevents this class of error entirely.

Designing Role-Bounded Tool Sets

Agent Role	Appropriate Tools	Tools to Exclude
Search agent	WebSearch, WebFetch	Write, Edit, Bash, database write tools
Document analyst	Read, Grep, Glob	Write, Edit, Bash, web tools

Agent Role	Appropriate Tools	Tools to Exclude
Code reviewer	Read, Grep, Glob	Write, Edit, Bash (read-only review)
Test runner	Bash, Read	Write, Edit, WebSearch
Synthesis agent	None (works from subagent outputs)	All external tools
Customer support agent	Customer lookup, order lookup, refund tool	Database admin, file system tools

allowedTools in the Agent SDK

The `allowedTools` parameter restricts which tools a subagent can access. When you create an agent definition, you specify the list of tools it can use:

```
agent = Agent(  
    name="search_agent",  
    description="Searches the web for current information",  
    allowedTools=["WebSearch", "WebFetch"],  
    prompt="Search the web for the given query and return relevant findings."  
)
```

This means the search agent can only use WebSearch and WebFetch. Even if the parent coordinator has access to Write, Edit, and Bash, the search agent cannot use them.

Permission Inheritance in Multi-Agent Systems

Anthropic's documentation states that permissive modes like `bypassPermissions` are inherited by subagents and cannot be relaxed per subagent. This has an important implication: if the coordinator runs with broad permissions, every subagent also runs with broad permissions unless you explicitly restrict them with `allowedTools`.

This means `allowedTools` is the primary mechanism for controlling subagent tool scope. You cannot rely on the system's permission mode to restrict subagents, as you must restrict their tools explicitly.

EXAM TIP: When a question describes a subagent that performed an action outside its intended scope the answer is to restrict its tools with `allowedTools`. When the question notes that the coordinator uses permissive mode the answer emphasizes that permissive modes are inherited, making `allowedTools` the only way to constrain subagents.

WORKED EXAMPLE — Tool Distribution in a Research System

A multi-agent research system has four agents:

- 1. Coordinator: delegates, synthesizes, evaluates coverage
- 2. Web Search Agent: finds current web sources
- 3. Document Analyst: reads and summarizes uploaded documents

4. Synthesis Agent: merges findings into a final report

Correct tool distribution:

- Coordinator: Agent tool (to spawn subagents). No direct data tools.
- Web Search Agent: `allowedTools = ["WebSearch", "WebFetch"]`. Cannot read local files or write anything.
- Document Analyst: `allowedTools = ["Read", "Grep", "Glob"]`. Cannot access the web or modify files.
- Synthesis Agent: `allowedTools = []`. Works entirely from the outputs passed by the coordinator. No tools needed.

Why this works: each agent can only do what its role requires. The search agent cannot accidentally modify files. The document analyst cannot make web requests. The synthesis agent has no tool access at all because it operates on pre-gathered data.

When to Use Tool Restriction vs. When Not To

- **Always restrict subagents:** The coordinator delegates specific tasks, while the subagent should have only the tools for that task.
- The coordinator may need broad access if it directly interacts with tools between delegations.
- **Interactive Claude Code sessions:** may need broad access because the developer is making decisions about what tools to use.
- **CI/CD pipeline invocations:** should restrict tool access to match the pipeline step's purpose.

Common Mistakes

- Giving every subagent every tool "for flexibility" increases decision complexity and risk.
- Relying on prompt instructions to prevent tool misuse instead of restricting tools with `allowedTools`.
- Assuming subagents inherit the coordinator's tool restrictions, they inherit permissions, not restrictions, unless you set `allowedTools`.
- Forgetting that the Agent tool itself should not be given to subagents (subagents cannot spawn their own subagents).

References

- <https://code.claude.com/docs/en/agent-sdk/tools>
- <https://code.claude.com/docs/en/agent-sdk/agent-loop>
- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/overview>

E. The tool_choice Setting

The `tool_choice` parameter controls whether Claude calls a tool and which one. It is the mechanism for moving from "Claude might call a tool" to "Claude definitely calls this specific tool." Getting this right is what separates a system that sometimes uses tools from one that always behaves predictably.

This section overlaps with Domain 4's structured output coverage but focuses specifically on `tool_choice` as a control mechanism for tool behavior, rather than on schema design for extraction

quality.

The Four Modes

Mode	Behavior	Use When
auto	Claude decides what to call a tool	General-purpose agents where tool use is sometimes needed
any	Claude must call a tool, but picks which	Multiple extraction schemas for different input types
tool	Claude must call a specific named tool	A specific step that must always run (e.g., metadata extraction)
none	Claude cannot call any tool	Testing text-only behavior, or disabling tools for a specific turn

Choosing the Right Mode

Use auto: when the task may or may not need a tool. A conversational agent that sometimes looks up orders and sometimes answers general questions should use auto mode. In this mode, Claude decides based on the request whether a tool call is appropriate.

Use any: when you need structured output but the input type varies. If you have multiple extraction schemas, such as one for invoices, one for receipts, and one for contracts, setting `tool_choice` to any forces Claude to call one of them, and Claude picks the right schema based on the document type.

Use tool: when a specific step must always execute. If every request must start with metadata extraction before anything else, setting `tool_choice` to the tool with the metadata extraction tool guarantees it runs first.

Use none: when you want text-only behavior for a specific turn, such as generating a summary from data already in context without calling any additional tools.

tool_choice and Structured Output

For extraction tasks, `tool_choice` is what guarantees Claude returns structured data instead of free-form text.

- **auto** allows Claude to respond with text instead of calling the extraction tool, which means sometimes you get prose instead of JSON.
- **any or tool** guarantees Claude calls the extraction tool, producing structured output every time.
- **Combining tool with strict: true** guarantees both that the tool is called AND that the inputs match the schema exactly.

WORKED EXAMPLE — Guaranteeing Structured Output

A pipeline extracts invoice data. The extraction tool has a defined JSON schema.

With `tool_choice: auto` → Claude sometimes responds with text: "I found the invoice number is INV-1234." This breaks the downstream parser.

With `tool_choice: {type: "tool", name: "extract_invoice"}` → Claude always calls the extraction tool with structured input. The parser always receives valid JSON.

With `strict: true` added → Claude's inputs to the tool are guaranteed to match the schema. No missing required fields, no wrong types.

Syntax Errors vs. Semantic Errors

The combination of `tool_choice` + strict tool use eliminates syntax errors (malformed JSON, missing fields, wrong types) but does NOT eliminate semantic errors (wrong values, inconsistent data, values in the wrong field). This distinction is heavily tested across Domains 2 and 4.

Error Type	Example	Caught by Schema?
Syntax: missing field	Required field invoice_number absent	Yes (with strict)
Syntax: wrong type	total_amount is a string instead of a number	Yes (with strict)
Semantic: wrong value	total_amount is 150.00 but line items sum to 180.00	No
Semantic: value in wrong field	Customer name appears in the email field	No
Semantic: contradicting fields	Status is "completed" but completion_date is null	No

Semantic errors require a validation layer (Pydantic, custom checks), as they cannot be caught by schema enforcement alone; this is covered in depth in Domain 4.

EXAM TIP: When a question asks how to guarantee both that a tool is always called AND that its inputs match the schema, the answer is `tool_choice: tool` (or any) combined with `strict: true`. When the question then asks about ensuring the VALUES are correct, that requires a separate validation layer. Schema enforcement handles structure; validation handles meaning.

Common Mistakes

- Using auto when you need guaranteed structured output, Claude may respond with text instead of calling the tool.
- Confusing any (model picks which tool) with tool (model must call a specific named tool).
- Assuming strict tool use guarantees correct values, it only guarantees correct structure.
- Setting `tool_choice` to none and wondering why tools are not called.

References

- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/overview>
- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/implement-tool-use>
- <https://docs.anthropic.com/en/docs/build-with-claude/structured-outputs>

F. MCP Server Architecture and Integration

The Model Context Protocol (MCP) is an open-source standard that allows Claude to connect with external tools, data sources, databases, APIs, and other systems. MCP servers extend Claude's built-in capabilities by exposing tools, resources, and prompts that Claude can use during a conversation. Understanding MCP is essential for Domain 2 because it is the primary mechanism for giving Claude access to the outside world beyond its built-in tools.

MCP servers can be configured at two scopes: project-level (shared with the team) and user-level (personal). The configuration determines which servers are available, how they authenticate, and whether they are committed to version control.

MCP Architecture

An MCP server is a separate process that Claude communicates with through the MCP protocol. The server exposes capabilities in three categories:

Capability	What It Is	How Claude Uses It
Tools	Callable functions (query database, create ticket, etc.)	Claude calls them like any other tool
Resources	Read-only content catalogs (schema descriptions, doc hierarchies)	Claude reads them to decide which tool to call
Prompts	Pre-defined prompt templates (deploy checklist, incident response)	Developers invoke them as slash commands

The key distinction: tools perform actions, resources provide information, and prompts are templates for common workflows. Tools are operational. Resources are informational. Prompts are workflow shortcuts.

Project-Level Configuration (.mcp.json)

The `.mcp.json` file sits at the project root and is committed to version control. Every team member who works on the project gets the same MCP servers. This is the correct scope for shared tools that the entire team needs.

```
{
  "mcpServers": {
    "venue-lookup": {
      "command": "node",
      "args": ["./.mcp-servers/venue-lookup/index.js"],
      "env": {
        "API_KEY": "${VENUE_API_KEY}"
      }
    }
  }
}
```

The `${VENUE_API_KEY}` is expanded from each developer's environment at runtime. The actual API key never appears in version control.

User-Level Configuration (~/.claude.json)

The `~/.claude.json` file is personal to each developer and not shared with the team. Use this for experimental or personal MCP servers.

Configuration	Location	Shared?	Use For
<code>.mcp.json</code>	Project root	Yes (version controlled)	Team-shared tools (databases, APIs, services)
<code>~/.claude.json</code>	User home directory	No (personal)	Experimental or personal tools

Environment Variable Expansion for Secrets

MCP configuration supports environment variable expansion using the `${VAR_NAME}` syntax. This is the correct way to handle API keys, database credentials, and other secrets in MCP configuration. Hardcoding secrets in `.mcp.json` would commit them to version control, exposing them to everyone with repository access. Environment variable expansion lets each developer store their credentials locally while sharing the server configuration.

MCP Resources for Cross-Server Efficiency

When Claude connects to multiple MCP servers (an issue tracker, a documentation wiki, and a database), it needs to know which server to query for each piece of information. Without this knowledge, Claude makes excessive exploratory calls, querying each server in turn until it finds the right one. MCP resources solve this by giving Claude a catalog of what each server contains. Each server exposes a description of its data (issue categories, document hierarchy, database schema), and Claude reads these catalogs to make informed decisions about which server to query.

Without Resources	With Resources
Claude queries Server A → no results. Queries Server B → no results. Queries Server C → found it. 3 calls, 2 wasted.	Claude reads catalogs → knows Server C has billing data → queries Server C directly. 1 call, 0 wasted.

MCP Prompts as Slash Commands

When an MCP server exposes prompts, Claude Code surfaces them as slash commands with the naming convention `/mcp_<servername>_<promptname>`. They are invoked explicitly by developers, and they do not auto-load into context, are not added to the tool registry, and are not surfaced as resources.

EXAM TIP: When a question asks how MCP prompts are accessed in Claude Code, the answer is `/slash` commands. When a question describes Claude making too many exploratory calls across multiple MCP servers, the answer is MCP resources (content catalogs). When a question asks about shared vs. personal MCP configuration, shared goes in `.mcp.json`, personal goes in `~/.claude.json`.

Common Mistakes

- Hardcoding API keys in `.mcp.json` instead of using environment variable expansion.

- Putting personal MCP servers in `.mcp.json` where they will be committed to version control.
- Confusing MCP resources (read-only catalogs) with MCP tools (callable functions).
- Expecting MCP prompts to auto-load into context when they are strictly slash commands.
- Not exposing MCP resources when connecting Claude to multiple servers, leading to excessive exploratory calls.

References

- <https://docs.anthropic.com/en/docs/agents-and-tools/mcp>
- <https://docs.anthropic.com/en/docs/claude-code/settings>
- <https://code.claude.com/docs/en/agent-sdk/mcp>

G. MCP Error Patterns and Tool Result Design

How you design the results that tools return to Claude is as important as how you design the tools themselves. A tool result that is too verbose consumes unnecessary context. A tool result that is too sparse gives Claude nothing to reason over. A tool result that returns an error without the `is_error` flag may be misinterpreted as valid data. Thus, getting the result design right is a reliability concern.

The `is_error` Flag

In MCP, tool results can be marked as errors using the `is_error` flag. When this flag is set to true, Claude understands that the tool call failed and adapts its behavior, it may retry, try a different approach, or report the issue.

Without the `is_error` flag, Claude may interpret error text as if it were valid data. If a database query times out and the tool returns `{"message": "connection timed out"}` without `is_error: true`, Claude may try to extract data from the error message, producing nonsensical output.

Designing Effective Tool Results

Tool results should be concise, structured, and relevant. The principle is to return only what Claude needs for its next reasoning step, not everything the backend produces.

Principle	Why	Example
Return only relevant fields	Reduces context consumption	Return 5 relevant order fields, not 40 raw database columns
Use structured format	Easier for Claude to extract specific values	JSON with named fields, not a prose paragraph
Include status information	Lets Claude know if the result is complete	"status": "complete" vs. "status": "partial, 3 of 5 records returned"
Filter before returning	Prevents context bloat from verbose backends	Filter the raw API response to include only the fields the agent needs

Verbose vs. Concise Tool Results

Concept	Best Used For	Key Benefit	Common Exam Trap
Verbose result	Debugging, one-off exploration	Contains all available data	Fills context window quickly, crowding out task context
Concise result	Production agents, long sessions	Preserves context budget	May miss data that turns out to be relevant later

The correct default is concise results for production agents. If Claude needs additional detail, it can make a follow-up tool call with a more specific query.

WORKED EXAMPLE — Tool Result Filtering

A customer lookup tool queries a database that returns 40 fields per record: internal IDs, audit timestamps, legacy migration flags, system metadata, etc.

Verbose (bad for production): Return all 40 fields. Each lookup consumes ~800 tokens. After 5 lookups, 4,000 tokens of the context window are filled with data Claude does not need.

Concise (good for production): Filter to 6 relevant fields: `customer_id`, `name`, `email`, `plan_type`, `account_status`, `created_date`. Each lookup consumes ~120 tokens. After 5 lookups, only 600 tokens consumed. The concise version preserves context for reasoning while providing everything the agent needs for the current task.

When to Transform Tool Results

Sometimes the raw tool output is structured but not in a form that Claude can reason about easily. Transformation reshapes the data for better reasoning.

- **Flatten nested structures** when Claude only needs one level of detail.
- **Aggregate multiple records into a summary** when the agent needs counts or trends, not individual records.
- **Convert codes to labels** when the raw data uses internal codes that Claude would need to look up.
- **Add computed fields** when the agent needs a derived value (like `"days_since_last_order"`) that the raw data does not include.

Common Mistakes

- Returning raw database responses with dozens of irrelevant fields.
- Not setting `is_error: true` on failed tool calls, causing Claude to interpret error messages as data.
- Returning a verbose tool results in long-running sessions, accelerating context rot.
- Transforming results so aggressively that Claude cannot verify the original data if needed.

References

- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/best-practices>
- <https://docs.anthropic.com/en/docs/agents-and-tools/mcp>

H. Built-in Tool Selection and Usage Patterns

Claude Code provides a set of built-in tools for interacting with codebases and file systems. Choosing the right tool for each task is a practical skill the exam tests directly. The built-in tools cover file operations (Read, Write, Edit), search (Grep, Glob), and execution (Bash). Understanding when to use each and how they interact is essential for both efficient exploration and correct exam answers.

Built-in Tool Reference

Tool	What It Does	Use When	Does NOT
Read	Loads file contents into context	Need to understand a file's logic, structure, or content	Modify files
Write	Creates or overwrites a file entirely	Creating new files, or fallback when Edit fails	Make targeted changes (use Edit)
Edit	Targeted old_string → new_string replacement	Making specific changes to existing files	Work when old_string is not unique
Grep	Searches file contents for patterns	Looking for specific text: imports, error messages, function calls	Search by file name (use Glob)
Glob	Searches for files by name/path	Finding files by naming convention or location	Search file contents (use Grep)
Bash	Executes shell commands	Running tests, git ops, installs, system info	N/A: can do almost anything

Grep vs. Glob: Content Search vs. Name Search

This is a simple but frequently tested distinction.

- **Grep** searches inside files, the content. Use when you know what text to find.
 - "Find all files that import `@company/auth`" → Grep
 - "Find where the error message `SYNC_CONFLICT` is defined" → Grep
 - "Find all callers of the `processRefund` function" → Grep
- **Glob** searches for files by name, the path. Use when you know the naming convention.
 - "Find all files named `cache*.py`" → Glob
 - "Find all test files matching `*.test.ts`" → Glob
 - "List all files in the `errors/` directory" → Glob

EXAM TIP: When a question asks to "find all files that import a specific package," Grep (content search). When a question asks to "find files named cache-something," Glob (name search). When a question asks to find an error message across services, Grep for the distinctive error text.

The Read → Write Fallback Pattern

The Edit tool requires its `old_string` parameter to be unique in the file. When a file has highly repetitive content (duplicate docstrings, repeated patterns, or identical structural blocks), Edit may fail because it cannot find a unique match.

The reliable fallback is the following:

- 1. Read the file to load its contents.
- 2. Modify the content in your reasoning (add the new function, change the target section).
- 3. Write the updated file.

This is less elegant than Edit but always works regardless of content repetition.

WORKED EXAMPLE — Edit Fails on Repetitive Content

A 150-line configuration file has many identical structural blocks. A developer asks Claude to insert a new block between two existing blocks. Claude tries Edit, but the `old_string` matches multiple locations.

Wrong approach: Use a very long `old_string` hoping to make it unique. This is fragile and may still fail.

Wrong approach: Append to the end of the file with Bash heredoc. This puts the block in the wrong location.

Correct approach: Read → modify → Write. Read the file, identify the correct insertion point, construct the complete updated content, and Write the new version.

Tool Selection for Common Tasks

Task	Primary Tool	Strategy
Understand a file's logic	Read	Load and analyze contents
Create a new file	Write	Write entire content
Change one specific line	Edit	<code>old_string</code> → <code>new_string</code>
Change a section in a repetitive file	Read → Write	Read, modify, write back
Find files importing a package	Grep	Search contents for import statement
Find files by naming pattern	Glob	Search by name/path
Find an error message's source	Grep	Search for distinctive error text
Run tests	Bash	Execute test command
Check git history	Bash	Execute git log/diff
Map codebase structure	Glob + Read	Glob to find files, Read key files

Tool Selection in Codebase Exploration

When exploring an unfamiliar codebase, the right tool depends on what you are trying to learn:

Understanding architecture: Start with Glob to map the directory structure, then read key files (interfaces, base classes, entry points) to understand the design.

Finding all callers of a function: Read the function's module and any wrapper modules to identify all exported names, then Grep for each name across the codebase.

Tracing an error message: Grep for the distinctive text of the error message across the codebase, then Read the matching files to understand context.

Decomposing a large task: Glob to map the codebase structure, Grep to find patterns and dependencies, then create a prioritized plan for the most impactful areas.

Agent SDK Built-in Tools Beyond File Operations

The Agent SDK includes additional built-in tools beyond file operations:

Category	Tools	Purpose
File operations	Read, Write, Edit	Interact with files
Search	Grep, Glob	Find content and files
Execution	Bash	Run shell commands
Web	WebSearch, WebFetch	Search the web, fetch pages
Discovery	ToolSearch	Find and load tools on demand
Orchestration	Agent, Skill	Spawn subagents, invoke skills
User interaction	AskUserQuestion	Ask the user for input
Task tracking	TaskCreate, TaskUpdate	Create and manage tasks

ToolSearch is particularly important for Domain 2. Instead of loading every possible tool at session start (which consumes context with tool definitions), ToolSearch lets Claude discover and load tools on demand. This keeps the initial context lean and only loads tool definitions when they are actually needed.

EXAM TIP: When a question describes an agent with many tools where the tool definitions consume too much context, ToolSearch is the mechanism for on-demand tool loading. When a question asks about spawning subagents, the Agent tool is the mechanism. These are built-in tools, not custom tools or MCP tools.

Common Mistakes

- Using Grep when you should use Glob (searching for file names, not file contents).
- Using Edit on repetitive files where the old_string cannot be unique, use Read → Write instead.
- Loading all available tools at session start instead of using ToolSearch for on-demand loading.
- Running destructive Bash commands without validation, prefer Read and Edit for file modifications.

References

- <https://docs.anthropic.com/en/docs/claude-code>
- <https://code.claude.com/docs/en/agent-sdk/tools>
- <https://docs.anthropic.com/en/docs/claude-code/best-practices>

Worked Examples Across Domain 2

Worked Example: Tool Description Rewrite for a Support System

A customer support system has three tools. The current descriptions cause frequent misrouting.

Before (poor descriptions):

Tool	Description
get_customer	"Get customer data"
get_orders	"Get order data"
process_refund	"Process a refund"

Problems: "Get customer data" and "Get order data" are too similar. Claude cannot reliably distinguish between them when a user says "I need help with my account." The descriptions do not specify what each tool returns, what parameters it needs, or what it explicitly does NOT do.

After (clear descriptions with boundaries):

Tool	Description
get_customer	"Retrieve customer profile by customer_id or email. Returns: name, email, phone, plan_type, account_status, created_date. Use when the user asks about their account, profile, or personal information. It DOES NOT return order history, use get_orders for that."
get_orders	"Retrieve order history for a customer by customer_id. Returns: order_id, date, items, quantities, status, total. Use when the user asks about orders, deliveries, purchases, or shipping. It does NOT return profile data, use get_customer for that."
process_refund	"Process a refund for a specific order. Requires: order_id, refund_amount, reason. Only use AFTER verifying the order exists with get_orders and confirming refund eligibility. Refunds above \$100 require human approval, do NOT process these automatically."

Each description specifies what the tool returns, what parameters it needs, when to use it, and what it does NOT do. The disambiguation statements prevent overlap. The process_refund description includes a prerequisite (verify order first) and a policy constraint (amount limit).

Worked Example: Structured Error Flow in a Pipeline

A data enrichment pipeline has three stages: lookup → enrich → store. Each stage can fail in different ways.

Stage 1 — Lookup: Queries an external API for company data.

Possible failures:

Failure	Category	Retryable	Correct Response
API returned 503	service_unavailable	true	Retry with backoff
API returned 401	permission_denied	false	Escalate — credentials need update

Failure	Category	Retryable	Correct Response
API returned 200 with empty results	N/A (not an error)	N/A	Return valid empty result
API connection timed out	timeout	true	Retry once, then annotate gap

Stage 2 — Enrich: Takes lookup data and adds computed fields.

If Stage 1 returned a structured error, Stage 2 should NOT process it as data. It should propagate the error forward with context: "Enrichment skipped because lookup failed: [original error]." If Stage 1 returned a valid empty result, Stage 2 should handle the empty case gracefully rather than crashing on missing fields.

Stage 3 — Store: Writes the enriched data to a database.

If Stage 2 propagated an error from Stage 1, Stage 3 should NOT attempt to store partial or error data. It should log the propagated error and either retry the full pipeline or annotate the coverage gap.

The cascade without structured errors: Stage 1 times out → returns empty → Stage 2 processes empty as "no data" → Stage 3 stores a record with null fields → downstream reports show the company has no data. The truth is the lookup never succeeded.

The cascade with structured errors: Stage 1 times out → returns `{is_error: true, category: "timeout", retryable: true}` → Stage 2 sees the error and propagates it → Stage 3 sees the propagated error and logs it → the system retries → lookup succeeds → pipeline completes correctly.

Worked Example: Tool Distribution for a CI/CD Review System

A CI/CD pipeline uses Claude Code for automated code review. The review has two phases:

Phase 1 — Per-file review: Each changed file is reviewed individually for local bugs, security issues, and error handling.

Phase 2 — Cross-file integration review: The changed files are reviewed together for interaction bugs, data flow issues, and contract mismatches.

Tool distribution:

Agent	allowedTools	Reason
Per-file reviewer	Read, Grep	Can read files and search for patterns, but cannot modify code or run commands
Integration reviewer	Read, Grep, Glob	Same as per-file, plus Glob for finding related files by naming pattern
Neither agent	Write, Edit, Bash, Agent	Review agents should not modify code, run commands, or spawn subagents

Why read-only tools: The review agents need to understand the code but should never modify it. Restricting to read-only tools prevents a class of errors where the review agent accidentally "fixes"

something, which would bypass the PR review process.

Worked Example: MCP Resource Catalogs for Multi-Server Routing

A development team connects Claude to three MCP servers:

1. **Jira MCP Server:** exposes tools for querying and creating issues
2. **Confluence MCP Server:** exposes tools for reading and searching documentation
3. **Database MCP Server:** exposes tools for querying production data

Without MCP resources: When a developer asks, "What is the status of the authentication refactor?", Claude has no way to know which server has the answer. It queries Jira (finds a ticket), then Confluence (finds a design doc), then the database (finds no relevant data). Three calls, one wasted.

With MCP resources, each server exposes a resource describing its content:

- **Jira resource:** Contains issue tracking data: project tickets, sprints, epics, bug reports, and feature requests. Query by project key, assignee, status, or text search.
- **Confluence resource:** Contains team documentation: design docs, architecture decisions, runbooks, and meeting notes. Search by title, space, or content.
- **Database resource:** Contains production data: user records, transactions, and system metrics. Query by table name and SQL.

Claude reads the resources and determines: "Authentication refactor" is likely a project ticket (Jira) and a design doc (Confluence). It queries both relevant servers and skips the database. Two calls, zero wasted.

Worked Example: Parameter Description Impact on Input Quality

A date range parameter is defined without a description. Claude passes dates in inconsistent formats across calls.

Without parameter description:

```
// Call 1: "2026-01-15"  
// Call 2: "January 15, 2026"  
// Call 3: "01/15/2026"  
// Call 4: "15-01-2026"
```

All four are valid date strings. None are consistent. The backend must handle four formats.

With parameter description:

```
"date_range": {  
  "type": "object",  
  "properties": {  
    "start": {  
      "type": "string",  
      "description": "Start date in ISO 8601 format (YYYY-MM-DD).  
Inclusive."  
    }  
  }  
}
```

```
    },
    "end": {
      "type": "string",
      "description": "End date in ISO 8601 format (YYYY-MM-DD). Inclusive.
Maximum range: 90 days."
    }
  }
}
```

Now Claude consistently passes ISO 8601 dates because the parameter description specifies the format. Parsing ambiguity is eliminated.

Worked Example: ToolSearch for On-Demand Tool Loading

A development agent has access to 30 MCP tools across multiple servers. Loading all 30 tool definitions at session start consumes approximately 6,000 tokens of context, leaving less room for code, conversation, and reasoning.

Without ToolSearch: All 30 definitions load at session start. The system prompt + tool definitions consume 10,000+ tokens before the first user message. Claude must evaluate all 30 descriptions on every turn. Performance degrades as the session grows.

With ToolSearch: Only the core built-in tools (Read, Write, Edit, Grep, Glob, Bash) load at session start. When Claude needs a specific MCP tool (like querying the database or creating a Jira ticket), it uses ToolSearch to discover and load that tool on demand. The session starts lean, and tool definitions are loaded only when needed.

What this shows: ToolSearch is the mechanism for managing tool definition overhead. In sessions with many available tools, on-demand loading preserves context for the actual work.

Domain 2 Services Appendix

Tool Design Reference

Component	Purpose	Impact on Reliability
Tool name	Short identifier	Minor: Claude uses descriptions more than names
Tool description	Tells Claude when and why to use this tool	Major: primary signal for tool selection
Input schema	Defines accepted parameters	Major: determines if Claude can construct valid call
Parameter description	Tells Claude what to pass for each parameter	Moderate: reduces inconsistent inputs
Boundary statement	Defines what the tool does NOT do	Major: prevents misrouting to similar tools

tool_choice Reference

Mode	Behavior	Use For
auto	Claude decides whether to call a tool	General-purpose agents
any	Claude must call a tool, picks which	Multiple extraction schemas
tool	Claude must call a specific named tool	Mandatory extraction step
none	Claude cannot call any tool	Text-only response for one turn

Structured Error Response Reference

Field	Purpose	Example Values
is_error	Marks the result as an error	true / false
category	Type of failure	"timeout", "permission_denied", "validation_error", "not_found", "rate_limited", "service_unavailable"
description	What specifically happened	"Database connection timed out after 30s"
retryable	Whether the same call might succeed on retry	true / false

Error Category Recovery Reference

Category	Retryable?	Recovery Strategy
timeout	Yes	Retry with backoff
rate_limited	Yes	Wait, then retry
permission_denied	No	Escalate
not_found	No	Treat as valid empty if appropriate
validation_error	Sometimes	Fix input, then retry
service_unavailable	Yes	Try alternative source or annotate gap

MCP Configuration Reference

Element	Description
MCP tools	Callable functions exposed by MCP servers
MCP resources	Read-only content catalogs describing server data
MCP prompts	Server-defined prompts surfaced as /mcp__ slash commands
.mcp.json	Project-level configuration (shared, version controlled)

Element	Description
~/.claude.json	User-level configuration (personal, not shared)
\${VAR_NAME}	Environment variable expansion for secrets

Built-in Tool Reference

Tool	Searches	Use For
Read	N/A (loads file)	Understanding file contents
Write	N/A (creates/overwrites)	Creating files, fallback when Edit fails
Edit	N/A (modifies file)	Targeted changes using unique string match
Grep	File contents	Finding text patterns, imports, error messages
Glob	File names/paths	Finding files by naming pattern
Bash	N/A (executes commands)	Running tests, git operations, system commands
WebSearch	Web content	Searching the web for current information
WebFetch	Web pages	Fetching and parsing web page content
ToolSearch	Tool registry	Discovering and loading tools on demand
Agent	N/A (spawns subagent)	Delegating tasks to focused subagents

Agent SDK Tool Categories

Category	Tools	Purpose
File operations	Read, Write, Edit	Interact with files
Search	Grep, Glob	Find content and files
Execution	Bash	Run shell commands
Web	WebSearch, WebFetch	Search and fetch web content
Discovery	ToolSearch	On-demand tool loading
Orchestration	Agent, Skill	Spawn subagents, invoke skills
User interaction	AskUserQuestion	Ask the user for input
Task tracking	TaskCreate, TaskUpdate	Create and manage tasks

Domain 2: Tool Design & MCP Integration — Sample Questions

Question 1

Your developer productivity agent answers questions about customer transactions by querying a PostgreSQL database. During design review, a team member proposes letting Claude generate and

execute raw SQL directly against the database based on user questions.

What should the integration design include instead?

1. Expose the database connection directly so Claude can write and execute raw SQL against it based on user input.
2. Skip input validation to reduce latency and trust the model to generate safe queries in all cases.
3. Route all database access through a secure query interface that validates inputs and returns structured responses to the agent.
4. Ignore transaction boundaries and allow the agent to run queries without tracking or rolling back incomplete operations.

Correct Answer: 3

Explanation:

When an agent interacts with a database, it translates natural language questions into queries at runtime. Without a controlled interface between the agent and the database, the model's query output reaches the database driver unsanitized. Datadog Security Labs documented this exact failure in Anthropic's reference PostgreSQL MCP server implementation: the server directly concatenated unsanitized user input into SQL statements executed by the database driver without filtering or validation, creating a SQL injection path that allowed malicious queries to be embedded through ordinary user input.

The correct design routes all database access through a query interface that parameterizes inputs before execution, enforces the principle of least privilege by restricting the agent to read-only or scoped operations where possible, validates query structure against an allowed schema, and returns results as structured data that the agent can reason over safely. Anthropic's own reference implementation used a read-only constraint as a core safety guardrail, recognizing that granting agents write access without validation controls creates unacceptable data integrity risk.

Hence, the correct answer is: **Route all database access through a secure query interface that validates inputs and returns structured responses to the agent.**

The option that says: *Expose the database connection directly so Claude can write and execute raw SQL against it based on user input* is incorrect because arbitrary SQL execution on unsanitized input is a SQL injection vulnerability. A single malformed input reaching the database driver can exfiltrate records or modify data without restriction.

The option that says: *Skip input validation to reduce latency and trust the model to generate safe queries in all cases* is incorrect because model-generated SQL is not guaranteed safe on every invocation. Validation is primarily a structural control that operates independently of model behavior and cannot be substituted by trusting the model.

The option that says: *Ignore transaction boundaries and allow the agent to run queries without tracking or rolling back incomplete operations* is incorrect because multi-step write operations that fail mid-execution leave the database in a partially updated state with no recovery path. Transaction management is simply required to maintain data consistency.

References:

- <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

- <https://claudecertifications.com/claude-certified-architect/domains/claude-code-config>

Question 2

Your extraction pipeline has a generic `analyze_document` tool that handles text extraction, data point identification, summarization, and claim verification. Users report inconsistent behavior, sometimes it extracts data, sometimes it summarizes, and the output format varies.

Which of the following is the best approach to fix this?

1. Split `analyze_document` into separate, purpose-specific tools, each with a defined input/output contract for a single task.
2. Add a mode parameter to `analyze_document` like `mode='extract'` or `mode='summarize'` so the model can specify the desired behavior.
3. Improve the system prompt to specify when each behavior of `analyze_document` should be used.
4. Keep the single tool but add a few comprehensive, one-shot examples covering each use case.

Correct Answer: 1

Explanation:

In structured data extraction pipelines built with Claude, tool design is a primary driver of output consistency. When a single tool handles multiple overlapping responsibilities, text extraction, data point identification, summarization, and claim verification, the model has no reliable signal for which behavior to invoke for a given request. The result is exactly what the scenario describes: inconsistent behavior and variable output formats.

Accordingly, the Structured Data Extraction scenario identifies generic multi-purpose tools as a core anti-pattern. The correct fix is to decompose `analyze_document` into purpose-specific tools, each with a single clear responsibility and a defined input/output contract. For example:

- `extract_data_points` - accepts a document and a schema, returns structured field values
- `summarize_content` - accepts a document, returns a concise summary
- `verify_claim_against_source` - accepts a claim and a source document, returns a verification result

Each tool now has a description that unambiguously maps to one type of task. The model selects between them based on the user's intent rather than guessing which internal mode of a generic tool to invoke. Output format consistency follows naturally because each tool has a fixed, defined output contract, not a variable one that changes depending on how the model interprets the request.

This is the single responsibility principle applied to tool design: one tool, one job, one output shape.

Hence, the correct answer is: **Split `analyze_document` into separate, purpose-specific tools, each with a defined input/output contract for a single task.**

The option that says: *Add a mode parameter to `analyze_document` like `mode='extract'` or `mode='summarize'` so the model can specify the desired behavior* is incorrect because it preserves the fundamental problem, a single tool doing multiple things, while adding a layer of complexity. The model must now both select the tool and infer the correct mode value from the user's request. This only

increases ambiguity rather than reducing it, and the output schema remains inconsistent across modes. A mode parameter is a surface-level fix that does not address the root cause.

The option that says: *Improve the system prompt to specify when each behavior of analyze_document should be used* is incorrect because prompt-based guidance is probabilistic. The model may follow the instructions most of the time, but the underlying tool remains ambiguous. In edge cases or ambiguous requests, the inconsistent behavior will persist. System prompt instructions guide behavior. They do not enforce output contracts or guarantee consistent tool selection.

The option that says: *Keep the single tool but add a few comprehensive, one-shot examples covering each use case* is incorrect because few-shot examples just add token overhead without fixing the structural ambiguity in the tool definition. The model can still misinterpret an overloaded tool regardless of how many examples are provided, especially on novel or ambiguous inputs. Examples are most effective when the tool itself is already well-defined; they cannot compensate for a poorly scoped tool design.

References:

- <https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools>
 - <https://claudecertifications.com/claude-certified-architect/scenarios>
-

Additional Exam Guidance for Domain 2

How Domain 2 Connects to Other Domains

Domain 2 concepts appear throughout the exam because tools are the mechanism through which Claude interacts with the outside world. Understanding these connections helps you recognize Domain 2 patterns in questions that appear to belong to other domains.

Connection to Domain 1 (Agentic Architecture): The agentic loop relies on tools to take action. Tool design determines whether the loop produces reliable results. Tool distribution (`allowedTools`) is how you control what each subagent in a coordinator-subagent architecture can do. Structured error responses are what enable the coordinator to make recovery decisions.

Connection to Domain 3 (Claude Code Configuration): Built-in tools (Read, Write, Edit, Grep, Glob, Bash) are Claude Code's primary mechanisms for interacting with codebases. MCP server configuration (`.mcp.json`, `~/.claude.json`) determines which external tools are available. The `—system-prompt` vs. `—append-system-prompt` distinction in CI/CD affects whether Claude retains access to its built-in tools.

Connection to Domain 4 (Prompt Engineering & Structured Output): `tool_choice` controls whether Claude produces structured output or text. Strict tool use (`strict: true`) guarantees schema-valid tool inputs. Few-shot examples improve output format but do NOT fix tool misrouting, that requires better tool descriptions (a Domain 2 fix, not a Domain 4 fix).

Connection to Domain 5 (Context Management & Reliability): Verbose tool results fill the context window and accelerate context rot. Filtering tool results before returning them is both a Domain 2 design decision and a Domain 5 reliability measure. Structured error responses that distinguish access failures from valid empty results prevent downstream agents from producing confident but wrong output.

Common Exam Patterns in Domain 2

The exam uses several recurring patterns for Domain 2 questions:

- The "wrong tool selected" pattern:** Claude consistently calls the wrong tool. The answer is almost always to improve tool descriptions, add specificity, boundary statements, and disambiguation. Never few-shot examples, never temperature, and never renaming alone.
- The "generic error" pattern:** A coordinator cannot recover because the tool returned a generic error. The answer is structured error responses with category, description, and retryability.
- The "false empty" pattern:** A downstream agent reports "no data" when the source was actually unreachable. The answer is distinguishing access failures from valid empty results using the `is_error` flag.
- The "over-permissive agent" pattern:** A subagent performs actions outside its role. The answer is restricting its tools with `allowedTools`.
- The "text instead of JSON" pattern:** An extraction pipeline sometimes gets text instead of structured output. The answer is changing `tool_choice` from auto to tool or any.
- The "Grep vs. Glob" pattern:** A question asks which tool to use for a search task. Content search → Grep. File name search → Glob.

Decision Framework for Domain 2 Questions

When you encounter a Domain 2 question, use this framework:

1. Is the problem about Claude selecting the wrong tool? → Improve tool descriptions (specificity, boundaries, disambiguation).
2. Is the problem about error handling in tools? → Structured errors with category, description, retryability, and `is_error` flag.
3. Is the problem about an agent doing something outside its role? → Restrict tools with `allowedTools`.
4. Is the problem about inconsistent structured output? → Check `tool_choice` setting; use tool or any instead of auto.
5. Is the problem about MCP server configuration? → Shared in `.mcp.json`, personal in `~/.claude.json`, secrets via `${VAR_NAME}`.
6. Is the problem about choosing which built-in tool to use? → Content → Grep, names → Glob, modify → Edit (or Read → Write for repetitive files).

Key Distinctions the Exam Tests

Concept A	Concept B	The Distinction
Tool name	Tool description	Description is the primary signal for tool selection, not the name
MCP tools	MCP resources	Tools are callable functions; resources are read-only content catalogs

Concept A	Concept B	The Distinction
MCP prompts	MCP tools	Prompts are slash command templates; tools are callable functions
access failure	valid empty result	Failure means query did not execute; empty means query returned nothing
tool_choice: auto	tool_choice: tool	Auto allows text response; tool guarantees tool call
tool_choice: any	tool_choice: tool	Any lets Claude pick which tool; tool forces a specific tool
strict tool use	semantic validation	Strict guarantees structure; validation checks meaning
Grep	Glob	Grep searches contents; Glob searches file names
Edit	Read → Write	Edit needs unique match; Read → Write works on repetitive files
.mcp.json	~/.claude.json	Project-level (shared); user-level (personal)
allowedTools	permissions.deny	allowedTools restricts which tools; deny blocks specific actions
Few-shot examples	Tool descriptions	Examples fix output format; descriptions fix tool selection
Verbose results	Concise results	Verbose fills context quickly; concise preserves context budget

Worked Example: Full Domain 2 Scenario

Scenario: A customer support system has three tools: `lookup_customer` (retrieves customer profile), `lookup_orders` (retrieves order history), and `process_refund` (processes a refund). The system is experiencing three problems:

1. Claude sometimes calls `lookup_customer` when the user asks about orders.
2. When the order database times out, the agent tells the customer "you have no orders."
3. A subagent spawned for order investigation accidentally processes a refund.

Domain 2 analysis:

Problem 1 — Tool misrouting: The `lookup_customer` description is "Look up customer information." The `lookup_orders` description is "Look up orders." These descriptions overlap because "customer information" could include order history. Fix: Rewrite `lookup_customer` to "Retrieve customer profile data: name, email, plan, account status. Does NOT return order history, for that use lookup_orders." Rewrite `lookup_orders` to "Retrieve a customer's order history by customer_id. Returns order IDs, dates, items, and status. Does NOT return profile data, for that you can use lookup_customer."

Problem 2 — False empty result: The `lookup_orders` tool returns an empty array when the database times out. The agent interprets this as "no orders exist." Fix: Return a structured error when the database is unreachable: `{ "is_error": true, "category": "timeout", "description": "Order`

`database connection timed out after 10s", "retryable": true }`. The agent can now distinguish "no orders" from "could not check."

Problem 3 — Over-permissive subagent: The order investigation subagent has access to all tools including `process_refund`. Fix: Set `allowedTools` for the investigation subagent to `["lookup_customer", "lookup_orders"]`. It can investigate but cannot take action.

Correct architecture after fixes: Clear tool descriptions with boundary statements → structured error responses with `is_error`, `category`, `retryability` → role-bounded tool sets with `allowedTools`.

References for Domain 2: Tool Design & MCP Integration

All links reference official Anthropic documentation.

Tool Use Overview

- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/overview>

Tool Use Implementation

- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/implement-tool-use>

Tool Use Best Practices

- <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/best-practices>

Model Context Protocol

- <https://docs.anthropic.com/en/docs/agents-and-tools/mcp>

Structured Outputs

- <https://docs.anthropic.com/en/docs/build-with-claude/structured-outputs>

Claude Code Overview

- <https://docs.anthropic.com/en/docs/claude-code>

Claude Code Settings

- <https://docs.anthropic.com/en/docs/claude-code/settings>

Claude Code Best Practices

- <https://docs.anthropic.com/en/docs/claude-code/best-practices>

Agent SDK Tools

- <https://code.claude.com/docs/en/agent-sdk/tools>

Agent SDK MCP

- <https://code.claude.com/docs/en/agent-sdk/mcp>

Agent SDK Agent Loop

- <https://code.claude.com/docs/en/agent-sdk/agent-loop>

Agent SDK Custom Tools

- <https://code.claude.com/docs/en/agent-sdk/custom-tools>

Prompt Engineering Overview

- <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>

Reduce Hallucinations

- <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/reduce-hallucinations>

CCA-F Official Exam Page

- <https://clau.de/CCAF>